

Project Report: Movie & TV Show Tracker

Komal Joshi

Date: November 9, 2025

Introduction

This report documents the design, implementation, challenges, and key learnings from developing the Movie & TV Show Tracker, a React-based web application that integrates with the TMDB (The Movie Database) API to allow users to discover, search, and manage a personalized watchlist of movies and TV shows. The project emphasizes component-based architecture, API integration, responsive design, and performance optimization.

1. Implementation Summary

The Movie & TV Show Tracker is a feature-rich, responsive web application built with React.js. It leverages real-time data from TMDB and provides a seamless, interactive user experience.

Architectural Overview:

The application follows a modern, component-based architecture that promotes reusability and separation of concerns. The project is organized into logical folders for pages, components, hooks, context, and services. Client-side routing is implemented using React Router DOM, enabling smooth navigation without full-page reloads.

Core Feature Implementation:

- API Integration: A dedicated service module (`src/services/api.js`) using Axios manages all communication with the TMDB API. A custom hook, `useApiFetch`, standardizes API calls and provides built-in handling for loading and error states across the app.
- State Management: A hybrid strategy was employed:
 - Local State (`useState`) handles component-specific logic (e.g., search inputs).
 - Global State (`Context API`) manages shared data such as authentication and the watchlist. `WatchlistContext` and `AuthContext` simplify global state handling while avoiding prop drilling.
- Data Persistence: User data (watchlist and login status) is persisted via `localStorage`, allowing session continuity even after refreshing or closing the browser.
- User Interface & Styling: The UI was built using Tailwind CSS, following a mobile-first, responsive design approach. A dark, cinematic theme enhances visual appeal. Reusable components such as `MediaCard`, `LoadingSpinner`, and `ErrorMessage` maintain consistency throughout the app.
- Authentication & Protected Routes: A mock authentication system was implemented to demonstrate access control. The `ProtectedRoute` component leverages `AuthContext` to secure the `/watchlist` route, redirecting unauthenticated users to the login page.
- Performance Optimization: To enhance speed and efficiency:

- React.memo prevents unnecessary re-renders of pure presentational components.
- React.lazy and Suspense enable route-based code splitting, reducing initial bundle size and improving load performance.

2. Challenges Faced

Several challenges arose during development and were systematically addressed:

- Effective State Management: Choosing between useState and Context for shared data was a major decision. The Context API provided a clean, efficient solution for managing global data like authentication and watchlist without adding external libraries.
- Handling Asynchronous Operations: Fetching external data asynchronously required careful handling of loading and error states. This was streamlined by creating the reusable useApiFetch hook, which encapsulated try...catch...finally logic across all API calls.
- API Data Consistency: TMDB returns slightly different data fields for movies and TV shows (e.g., title vs name, release_date vs first_air_date). Defensive design in MediaCard ensured correct rendering across both types without breaking.
- Secure API Key Management: The TMDB API key was stored in an .env file and accessed via process.env to ensure it remained hidden from version control and public exposure.

3. Key Learnings

This project was a major learning milestone, deepening both conceptual understanding and practical skills in modern React development.

- Advanced React Hooks: Gained hands-on experience beyond the basics — using useContext, useParams, useNavigate, and custom hooks to create modular, maintainable code and avoid repetition.
- Component-Based Architecture: Learned to decompose the UI into small, focused components, leading to cleaner code, faster debugging, and improved scalability.
- Global State Management: Implemented a lightweight yet powerful global state solution using the Context API, gaining confidence in handling cross-cutting concerns like authentication and shared data without Redux.
- Responsive Design with Utility-First CSS: Built a mobile-first, responsive interface efficiently using Tailwind CSS, mastering how utility classes can quickly produce flexible layouts and adaptive designs.
- Performance Optimization: Realized that performance is an integral part of user experience. Implementing React.memo and route-based code splitting directly improved app speed and responsiveness.

4. AI-Assisted Development

AI assistance was responsibly integrated into the development process as a supportive tool rather than an author. It contributed by:

- Suggesting project structure best practices for React applications, leading to a clean, scalable folder organization.
- Helping resolve styling challenges in Tailwind CSS, such as responsive layouts and component alignment.
- Assisting in refining documentation and report clarity, ensuring the technical content was

concise, professional, and accessible.

All implementation and design decisions were developer-driven, with AI used strictly for guidance and efficiency enhancement.

Conclusion

Developing the Movie & TV Show Tracker was both a technically and creatively rewarding experience. It strengthened my understanding of React, API integration, state management, and UI design while demonstrating how thoughtful architecture and optimization can elevate a web application's quality and performance.

PS: I still need to practice more of this.

Thankyou